

AutoReproducer: 基于多智能体协作的论文 自动复现与优化系统

2026-09-02

一、项目概述

1.1 项目背景

近年来,人工智能研究越来越依赖于从论文到代码的复现、调试和迭代优化的漫长周期。研究工具分散在论文、代码仓库、数据集和未文档化的工程假设之间,使得大量的科研精力被重复性的实验迭代所消耗。论文复现危机已成为制约科研效率的核心瓶颈 – 据统计,大量顶会论文的代码无法在公开环境下直接运行,原因包括依赖版本冲突(bit-rot)、API 变更、缺失环境配置等。

2025 年 4 月, OpenAI 发布了 PaperBench 基准测试系统, 评估 AI 智能体自主复现前沿 AI 研究的能力。该系统选取了 20 篇 ICML 2024 Spotlight 和 Oral 论文, 构建了 8316 个可独立评估的复现子任务。测试结果显示, 表现最好的 Claude 3.5 Sonnet 仅达到 21.0% 的复现成功率, 而人类机器学习博士在相同子集上达到了 41.4% 的得分。这一结果清晰揭示了当前 AI 在自主科研复现方面与人类专家之间的显著差距。

与此同时, 多智能体系统与大语言模型的结合为这一问题的自动化解决提供了新的可能。ReproRun 采用“1个编排器+6个专职Agent”的架构实现了论文的自动复现; AutoSOTA 通过 8 个专业 Agent 协作, 成功在 105 篇顶会论文上发现了超越原始方法的新 SOTA 模型; Mimoso-AI 通过执行反馈持续优化多智能体工作流, 在 ScienceAgentBench 上达到 43.1% 的成功率。2025 年 3 月, Sakana AI 推出的 AI Scientist-v2 通过了 ICLR 会议研讨会的同行评审, 成为首篇由 AI 科学家撰写并通过同行评审的科研论文, 标志着 AI 在科研领域的重大突破。

核心洞察: 当前 AI 智能体在论文复现方面表现远低于人类专家(Claude 3.5 Sonnet 21% vs 人类博士 41.4%), 且现有系统在复现与优化之间割裂, 缺乏端到端闭环能力。这为构建融合复现与优化的自动化系统提供了明确的创新空间。

1.2 项目目标

本项目旨在设计和实现一个基于多智能体协作的论文自动复现与优化系统(AutoReproducer), 能够实现以下三大核心目标:

1. 自动复现: 输入论文标题或 PDF, 系统自动完成“读论文->找代码->搭环境->运行->比对结果”全流程
2. 智能优化: 在复现成功的基础上, 自动进行超参数调优、架构改进, 尝试超越原始论文的性能
3. 预算可控: 在有限的 API 调用和计算资源预算内完成研究和优化

1.3 核心价值

- 对科研的价值: 将论文复现从“手工艺”转变为“自动化流水线”, 释放科研人员创造力
- 对课程的价值: 完整实践多智能体系统设计、大模型应用、自动化工程等核心技术
- 对社区的价值: 构建完全开源的论文自动复现工具, 为更广泛的科研社区提供可复用的基础设施

二、现状调研

2.1 论文复现危机的现状

论文复现危机是当前科学界面临的核心挑战之一。根据 Stanford HAI 发布的 2025 AI Index Report, 2013 至 2023 年间 AI 相关发表论文数量从约 10.2 万篇增长至 24.2 万篇, 增长了近 2.4 倍, AI 论文占计算机科学论文的比例从 21.6% 上升到 41.8%。然而, 论文数量的爆发式增长并未伴随复现性的同步提升。

ReproNLP 系列共享任务(2020-2025 年共六届)持续追踪 NLP/ML 领域的可复现性, 其研究揭示了导致低复现率的深层原因: 实验设计差异、随机种子影响、数据处理流程不一致等。Foundation Model Transparency Index 显示, 主要模型开发者的平均透明度评分从 2023 年 10 月的 37% 提升至 2024 年 5 月的 58%, 但仍有大量改进空间。

在评估基准方面, 2024-2025 年出现了两个重要的科研自动化评估系统: PaperBench 和 ScienceAgentBench。PaperBench 由 OpenAI 联合论文原作者制定 8316 个评分子任务, 测试 AI 从零复现 ICML 论文的能力; ScienceAgentBench 从同行评议论文中提取跨学科任务(涵盖生物信息学、计算化学、地理信息科学、心理学等), 评估数据驱动科学发现中的语言 Agent 能力。

评估基准	任务规模	最佳 AI 表现	人类基线	核心发现
PaperBench	20 篇 ICML 论文, 8316 子任务	Claude 3.5 Sonnet 21.0%	ML 博士 41.4%	AI 在长程任务执行上存在显著弱点
ScienceAgentBench	跨学科科研任务	32.4%(3 次尝试)	34.3%(专家辅助)	AI 可 10 分钟生成程序草稿, 但远未达到完全自动化
PaperBench Code-Dev	轻量级评估子集	GPT-4 43.4%	41.4%	代码开发场景下 AI 接近人类水平

表 1: 主要科研自动化评估基准对比

2.2 国内外研究现状

2.2.1 论文自动复现系统

系统	架构	核心特点	局限性
ReproRun	1 编排器+6 Agent	六步流水线, 4 篇论文全验证通过	仅优化复现流程, 无优化能力

系统	架构	核心特点	局限性
AutoReproduce	多智能体框架	论文谱系算法挖掘引用文献隐含知识	聚焦复现, 不涉及改进
RepLLM	4 个专业 Agent	面向网络系统研究, 2 小时复现 95% 基准	领域限定
HiRAS	层级式多智能体	监督管理者协调细粒度 Agent	纯复现, 无优化
Prompt-Free Agents	验证+优化双 Agent	无需人工设计提示词, 自动验证与改进, 在 PaperBench Code-Dev 上提升约 15%	仅改进代码生成质量

2.2.2 自动研究优化系统

系统	架构	核心能力	成果
AutoSOTA	8 个专业 Agent	端到端 SOTA 模型自动发现	发现 105 个新 SOTA, 平均 5 小时/篇
Sea-Mult-Agent	Intent Router+Planner+DAG	预算受限的自动研究	真实执行后 Keep/Reject 机制
Mimosa-AI	自演化多智能体	达尔文式工作流优化	ScienceAgentBench 43.1% 成功率
AI Scientist v2	LLM 驱动科研 Agent	端到端科研: 选题->实验->写作	首篇 AI 论文通过 ICLR 同行评审(2025.3)

2.2.3 多智能体协作框架生态

2024-2025 年, 多智能体协作框架生态快速发展。微软的 AutoGen(GitHub 38.6K stars)支持创建和管理多个自主 Agent 协同完成复杂任务; LangGraph(8.6K stars)基于 LangChain 引入有向循环图理念, 支持多智能体编排; OpenAI Swarm(18.4K stars)提供轻量级多智能体编排; CrewAI(8.6K stars)专注复杂任务的分布式协同与动态角色分配。此外, Scaling Multi-Agent Collaboration(arXiv:2406.07155)探索了多智能体协作系统的规模定律, 研究持续增加智能体数量是否能带来性能提升。

2.3 现有工作不足

1. 复现与优化割裂: 现有系统或只做复现(ReproRun、RepLLM), 或只做优化(AutoSOTA), 缺乏从复现到优化的端到端闭环
2. 预算管理缺失: 多数系统不考虑 API 调用成本和计算资源限制, 不适合课程项目场景
3. 验证机制薄弱: 现有框架缺乏对每个生成步骤输出的验证和优化机制。Prompt-Free Collaborative Agents(arXiv:2512.02812)的研究表明, 引入验证-优化双 Agent 可在 PaperBench Code-Dev 和 Paper2CodeBench 上分别提升约 15% 和 13%, 但该方案尚未与完整复现流程集成
4. 可解释性不足: 优化过程的决策缺乏可审计的记录, 难以复现和改进
5. 长程任务执行能力弱: PaperBench 测试表明, 多数模型会提前终止任务, 声称已完成或遇到不可解决的问题, 实际未能制定和执行有效的复现策略

2.4 本项目定位

基于以上分析,本项目定位为:一个融合复现与优化的、预算可控的、具备验证机制的多智能体论文自动研究系统。与现有系统相比, AutoReproducer 的核心差异化在于将复现与优化统一为闭环架构,并在有限预算约束下通过 UCB 算法智能分配资源,同时通过 Prompt-Free 双层验证机制保障每一步输出质量。

三、创新点

3.1 创新点一:复现-优化一体化的端到端闭环架构

描述: 现有系统要么只做复现(ReproRun), 要么只做优化(AutoSOTA)。本项目首次将两者融合为一个闭环系统 – 复现模块验证论文可复现性后, 自动将结果传递给优化模块; 优化模块提出改进方案并实验验证; 改进结果反馈回知识库, 为后续复现提供经验。

技术实现: 设计统一的工作流状态机, 定义从 INIT -> PAPER_READ -> CODE_FOUND -> ENV_BUILT -> REPRODUCED -> OPTIMIZING -> OPTIMIZED -> DONE 的完整状态流转, 每个状态由对应的 Agent 负责推进。该设计借鉴了 ReproRun 的 Loop Engineering 和 Mimosa-AI 的审计追踪理念, 但首次将优化阶段纳入同一状态机。

3.2 创新点二: 预算感知的智能资源调度

描述: 借鉴 Sea-Mult-Agent 的预算受限设计思想, 引入预算感知调度器, 在有限的 API 调用次数和计算资源内, 使用 UCB(Upper Confidence Bound)算法在不同优化方向间智能分配预算。

技术实现: UCB 算法是多臂老虎机(Multi-Armed Bandit)问题的经典解决方案, 通过平衡探索(Exploration)与利用(Exploitation)来最大化累积收益。系统启动时设定总预算(如 100 次 API 调用、2 小时 GPU 时间), Scheduler 维护各优化方向的“价值估计”, 采用 UCB 策略选择下一轮尝试的方向。Budget-Constrained MAB 理论证明, UCB 算法在预算约束下可实现 $O(NK^4 \log B)$ 的遗憾界(regret bound), 为本项目的预算效率提供理论保障。达到预算上限或目标性能即停止。

3.3 创新点三: Prompt-Free 的双层验证-优化机制

描述: 借鉴 Prompt-Free Collaborative Agents(arXiv:2512.02812)的思想, 无需人工设计额外的验证提示词, 直接复用各 Agent 的系统提示词作为质量检查标准。设置验证 Agent 和优化 Agent 双层协作: 验证 Agent 检查输出质量, 优化 Agent 基于问题改进。该方案在 PaperBench Code-Dev 和 Paper2CodeBench 上分别实现了约 15% 和 13% 的性能提升。

技术实现: 每个 Agent 在执行任务后, 验证 Agent 读取该 Agent 的系统提示词作为标准, 检查输出是否满足要求; 若不满足, 优化 Agent 基于差异进行修正, 形成“生成->验证->修正->再验证”的闭环。与 Self-Refine 等需要人工设计提示词的方法相比, 该方案具有更强的适应性和扩展性。

3.4 创新点四: 可审计的完整实验追踪

描述: 参考 ReproRun 的 Loop Engineering 设计和 Mimosa-AI 的审计追踪理念, 构建完整的实验审计日志系统, 记录每一次代码尝试、环境配置、参数调整和结果比对, 确保整个研究过程完全可复现、可审计。

技术实现: 所有 Agent 的输入输出、决策依据、执行结果以结构化 JSON 格式写入 `experiment_ledger/` 目录, 支持按时间轴回放整个研究过程。这与 PaperBench 的层级化评分标准 (8316 个可独立评估子任务) 设计理念一致, 为系统提供了细粒度的过程追踪能力。

四、技术路线

4.1 总体架构

系统采用“1个编排器(Orchestrator)+7个专职Agent”的多智能体协作架构。Orchestrator 负责任务分解、流程控制、状态管理和预算调度; 7个 Agent 分为两层: 第一层包含 Paper Reader、Resource Finder、Env Builder、Code Executor 和 Result Validator, 负责论文复现全流程; 第二层包含 Optimizer 和 Verifier, 负责智能优化和质量验证。

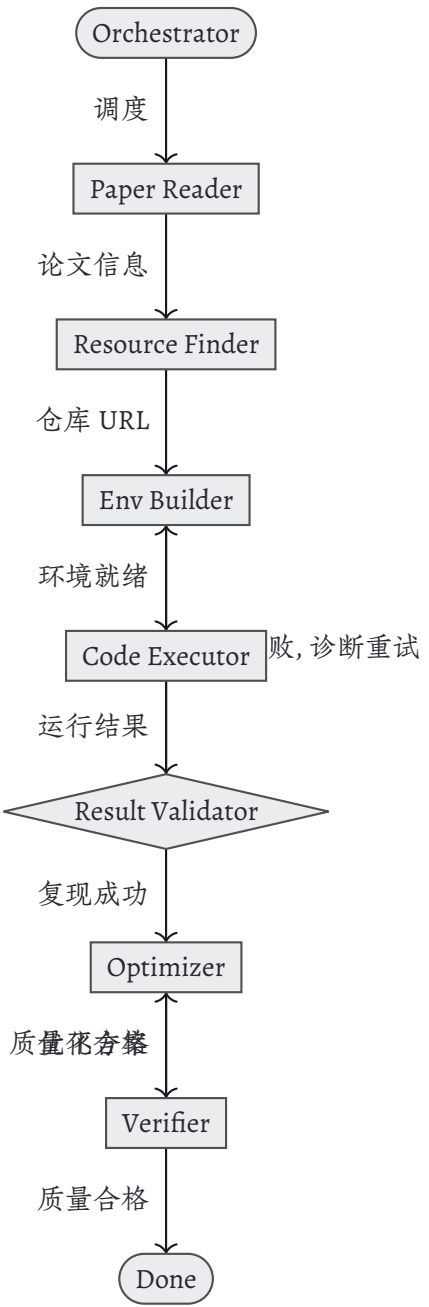


Figure 1: AutoReproducer 系统架构与 workflow

4.2 各 Agent 职责

Agent	职责	输入	输出
Paper Reader	解析论文 PDF, 提取方法、依赖、数值声明	论文 PDF/标题	结构化论文信息 JSON
Resource Finder	查找代码仓库、数据集	论文信息	仓库 URL、数据源列表
Env Builder	自动构建可执行环境, 处理依赖冲突	仓库 URL	可执行环境+依赖报告
Code Executor	在沙箱中安全运行代码	环境+代码	运行日志+结果
Result Validator	比对实际结果与论文声明	运行结果+论文声明	复现报告(成功/失败+差异)
Optimizer	提出并尝试优化方案	复现结果	优化方案+改进结果

Agent	职责	输入	输出
Verifier	检查各步骤输出质量	任意 Agent 输出	质量报告+修正建议

4.3 核心技术栈

层级	技术选型	理由
LLM 后端	Ollama + Qwen2.5/DeepSeek-R1(量化版)	本地部署, 零 API 成本, 普通笔记本可运行
多智能体框架	自研轻量级框架(基于 LangChain 或自建)	灵活可控, 适合学习理解; 参考 AutoGen /LangGraph 设计
代码执行沙箱	Docker + Python 虚拟环境	隔离安全, 环境可重现
依赖解析	pip/conda + 自定义依赖图分析	处理版本冲突
前端展示	Streamlit/Gradio	快速构建交互式 Demo
数据持久化	SQLite + JSON Ledger	轻量级, 完整审计追踪

4.4 核心工作流程

4.4.1 Phase 1: 论文解析(Paper Reader)

- 输入论文标题或 PDF -> 自动搜索/读取 PDF
- 提取: 模型架构、数据集、超参数、依赖列表、声称的数值结果
- 输出结构化 JSON, 作为后续所有步骤的“真相来源”

4.4.2 Phase 2: 资源定位(Resource Finder)

- 根据论文信息在 GitHub/PMC 搜索代码仓库
- 识别依赖文件(requirements.txt, environment.yml, setup.py 等)
- 输出仓库 URL、依赖列表、数据源

4.4.3 Phase 3: 环境构建(Env Builder)

- 自动创建 Docker 镜像或 Python 虚拟环境
- 逐项安装依赖, 处理版本冲突
- 5 轮依赖诊断循环: 分类错误 -> 定点修复 -> 重验证
- 输出可执行环境+依赖修复报告

4.4.4 Phase 4: 代码执行(Code Executor)

- 在沙箱中运行代码(smoke test + full run)
- 捕获标准输出、错误日志、生成的数值结果
- 输出运行日志+结果数据

4.4.5 Phase 5: 结果验证(Result Validator)

- 将实际运行结果与论文声称数值进行比对
- 输出复现报告: 成功/失败、数值差异、可能原因分析
- 判断标准: 环境是否 OK、输出是否 OK、描述是否 OK

4.4.6 Phase 6: 智能优化(Optimizer) [核心创新]

- 仅在复现成功后触发
- 分析论文方法, 提出可证伪的改进假设
- 尝试方向: 超参数调优、模块替换、架构调整
- 真实执行后根据结果 Keep/Reject
- 采用 UCB 算法在预算内分配尝试资源
- 输出优化报告: 尝试记录、最佳结果、改进幅度

4.4.7 Phase 7: 验证闭环(Verifier) [核心创新]

- 贯穿整个流程, 每个步骤完成后触发
- 复用各 Agent 的系统提示词作为质量标准
- 检查输出是否满足要求, 不满足则触发修正
- 输出质量报告+修正记录

4.5 数据流与状态管理

系统采用中心化状态机管理数据流转。用户输入提交至 Orchestrator 后, 状态机初始化; 每个 Phase 完成后状态推进至下一阶段, 同时触发 Verifier 进行质量检查, 并将所有输入输出写入 Ledger。状态流转路径: INIT -> PAPER_READ -> RESOURCE_FOUND -> ENV_BUILT -> REPRODUCED -> OPTIMIZING -> OPTIMIZED -> DONE。最终生成包含复现报告和优化报告的完整研究文档。

状态机设计的核心优势在于: (1)每个状态由明确指定的 Agent 负责, 职责清晰; (2)失败状态可回退至上一阶段进行重试; (3)Ledger 记录支持任意时间点的状态回放和审计。

五、可行性分析

5.1 技术可行性

维度	分析	结论
大模型部署	Ollama 支持 Qwen2.5-7B 等模型在普通笔记本(16GB 内存)上运行	可行
多智能体框架	已有 LangChain、AutoGen 等成熟框架可参考, 自研轻量级框架复杂度可控	可行
代码执行沙箱	Docker 提供成熟的容器隔离方案	可行
依赖解析	pip/conda 提供依赖管理 API, 版本冲突可通过降级解决	可行
论文解析	PyPDF2/PDFPlumber 提取文本, LLM 进行结构化信息抽取	可行
算力需求	本地量化模型+仅优化阶段调用 LLM, 普通笔记本即可满足	可行

5.2 数据可行性

- 论文数据: arXiv 等平台提供海量开源论文, 可选择 CV/NLP/ML 领域的公开论文
- 代码数据: GitHub 上大量论文官方代码仓库开源
- 数据集: CIFAR、MNIST、ImageNet 等标准数据集公开可用
- 评估数据: 已有 PaperBench(8316 个子任务)、ScienceAgentBench(跨学科任务)等公开基准

5.3 资源可行性

- 硬件: 普通开发笔记本(16GB 内存+GTX 1060 以上)即可运行
- 软件: 全部使用开源技术栈, 零授权成本
- 时间: 3 人 x14 周, 总计约 42 人周, 工作量可控
- 预算: 使用本地模型, 零 API 调用费用

5.4 风险与应对

风险	概率	影响	应对策略
部分论文代码无法复现	高	中	选择已有公开代码的论文; 复现失败也记录原因作为有价值输出
环境构建失败	中	中	设计 5 轮诊断循环; 降级到最小可用路径
LLM 输出不稳定	中	低	引入 Verifier Agent 进行质量检查; 多次采样
优化效果不明显	中	低	优化本身是探索性过程, 记录所有尝试即有学术价值
进度延误	中	中	采用敏捷开发, 每两周一个可演示的里程碑
长程任务执行提前终止	中	中	参考 PaperBench 发现, 设计目标驱动的终止判断机制, 区分“完成”与“预算耗尽”

六、三人协作分工

6.1 角色分工

角色	成员	核心职责	具体任务
A - 系统架构师	成员 1	整体架构设计、多智能体框架、编排器	设计 Agent 通信协议与状态机; 实现 Orchestrator 核心逻辑; 搭建 Docker 沙箱环境; 集成各模块
B - Agent 开发工程师	成员 2	各专业 Agent 实现、LLM 集成	实现 Paper Reader+Resource Finder; 实现 Env Builder+Code Executor; 实现 Result Validator; Ollama 集成与 prompt 工程
C - 优化与前端工程师	成员 3	优化模块、验证模块、前端展示	实现 Optimizer(含 UCB 调度); 实现 Verifier(Prompt-Free 验证); 构建 Streamlit 前端界面; 实验数据可视化与报告生成

6.2 协作机制

- 代码管理: Git + GitHub, 采用 Feature Branch 工作流
- 文档规范: 每个 Agent 有独立的 agent_xxx.md 指令文档
- 接口约定: Agent 之间通过结构化 Markdown/JSON 文件传递信息, 不共享对话上下文
- 周会制度: 每周一次同步会议, 汇报进展、协调接口、解决问题
- 集成测试: 每两周一次端到端集成测试, 确保各模块协同工作

七、时间规划与里程碑

7.1 总体时间线(14 周)

项目总周期 14 周, 分为需求分析、核心开发、复现流程、优化验证、集成测试和答辩准备六个阶段。

周次	阶段
W1-2	需求分析与技术预研
W3-5	核心框架与基础 Agent 开发
W6-8	完整复现流程实现与调试
W9-11	优化模块与验证模块开发
W12-13	系统集成与端到端测试
W14	文档完善与答辩准备

7.2 详细里程碑

阶段	周次	里程碑	交付物	负责人
开题准备	W1-2	完成开题报告	开题报告文档、技术选型确认	全员

阶段	周次	里程碑	交付物	负责人
核心框架	W3	多智能体框架搭建完成	Orchestrator 核心代码、Agent 基类	A
基础 Agent	W4-5	Paper Reader + Resource Finder 完成	2 个 Agent 可独立运行	B
环境与执行	W6-7	Env Builder + Code Executor 完成	可在 Docker 中运行论文代码	B
复现闭环	W8	完整复现流程打通	端到端复现 1 篇论文成功	A+B
优化模块	W9-10	Optimizer + Verifier 完成	可对复现结果进行优化尝试	C
前端展示	W11	前端界面完成	交互式 Demo 可展示全流程	C
集成测试	W12-13	系统端到端测试	3-5 篇论文的完整复现+优化报告	全员
答辩准备	W14	最终答辩	项目报告、PPT、Demo 视频	全员

7.3 关键检查点

- W5 检查点: 能否从一篇论文 PDF 中提取出结构化的方法、依赖和数值声明?
- W8 检查点: 能否自动复现一篇有公开代码的论文并生成复现报告?
- W11 检查点: 能否在复现基础上自动尝试至少 3 种优化方案?
- W13 检查点: 能否在 3 篇以上不同领域论文上验证系统有效性?

八、各阶段报告要点

8.1 开题报告(W2)

待解决的问题:

1. 论文复现为何如此困难?(依赖冲突、API 变更、环境差异等)
2. 现有自动复现系统为何无法同时做到复现+优化?
3. 如何在有限算力下实现有效的自动研究?

现状调研: 综述 ReproRun、AutoReproduce、RepLLM、AutoSOTA、Sea-Mult-Agent、Mimosa-AI 等系统, 分析各系统的架构特点、优缺点、适用范围, 明确本项目的差异化定位。参考 PaperBench 和 ScienceAgentBench 的评估方法设计本系统的验证方案。

预期成果: 一个可运行的论文自动复现与优化系统; 3-5 篇论文的完整复现+优化报告; 项目技术报告与演示 Demo。

8.2 中期报告(W8)

创新点进一步明确: 通过前期开发验证各创新点的技术可行性, 根据实际开发经验调整和细化创新点表述, 补充实验数据支撑创新点价值。

- Demo 要求:
- 能够端到端自动复现至少 1 篇论文
 - 展示完整的六步流程: 读论文 -> 找代码 -> 搭环境 -> 运行 -> 验证 -> 生成报告
 - 展示状态机流转和审计日志
 - 前端界面可展示中间步骤和结果

中期交付物: 系统核心框架代码(Orchestrator+ 至少 4 个 Agent); 1 篇论文的完整复现报告; 中期技术报告; Demo 演示视频(5 分钟)。

8.3 最终答辩(W14)

成果	说明	验收标准
完整系统	1 编排器+7Agent 的完整实现	所有 Agent 可协同完成从复现到优化的全流程
复现验证	3-5 篇论文的复现报告	至少 3 篇论文成功复现(含成功和失败案例)
优化案例	至少 2 个优化成功案例	优化后性能超过原始论文或发现有效改进方向
技术报告	完整项目文档	含架构设计、Agent 指令、实验数据、代码注释
审计日志	完整实验追踪记录	所有尝试可回溯、可复现
Demo 视频	10 分钟系统演示	展示完整工作流程和核心功能

答辩重点: 系统架构设计与创新点(40%); 实验验证与结果分析(30%); 技术难点与解决方案(20%); 项目总结与未来展望(10%)。

九、预期成果与评估指标

9.1 定量指标

指标	目标值	测量方式
论文复现成功率	>=60%(3/5 篇)	端到端复现成功数/总尝试数
复现平均耗时	<=30 分钟/篇	从输入到报告生成的总时间
优化提升幅度	>=3% 性能提升	优化后 vs 原始论文指标
预算控制	100 次 API 调用内完成	实际调用次数统计
代码执行率	>=80%	成功运行的代码行数/总代码行数

9.2 定性指标

- 系统可用性: 非技术人员能否通过简单输入使用系统

- 可扩展性: 添加新 Agent 是否简便
- 可解释性: 审计日志能否清晰展示每一步的决策依据
- 鲁棒性: 面对不同类型的论文(CV/NLP/ML)能否稳定工作

十、总结

本项目 AutoReproducer 以“自动复现+智能优化”为核心目标, 通过 1 个编排器+7 个专职 Agent 的多智能体协作架构, 构建了一个从论文输入到优化输出的端到端自动化研究系统。

项目的核心创新在于: 复现-优化一体化的闭环设计、预算感知的智能资源调度、Prompt-Free 的双层验证机制和完整的实验审计追踪。这些创新使得系统不仅能够复现论文, 还能在有限资源下自动探索改进方案。

三人团队分工明确、时间规划合理、技术路线成熟、风险评估充分, 项目具有高度的可行性。最终成果将是一个可运行的、有实际科研价值的、完全开源的论文自动复现与优化系统, 既可作为课程项目的优秀成果, 也可为更广泛的科研社区提供工具支持。

参考文献

- [1] OpenAI. (2025). PaperBench: Evaluating AI's Ability to Replicate AI Research. arXiv:2504.01848.
- [2] Chen, et al. (2024). ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery. arXiv:2410.05080.
- [3] Lin, Z., Cai, Q., Shen, L., Xiao, M. (2025). Enhancing Automated Paper Reproduction via Prompt-Free Collaborative Agents. arXiv:2512.02812.
- [4] Stanford HAI. (2025). The 2025 AI Index Report. <https://hai.stanford.edu/ai-index>.
- [5] Zhou, D.P., Tomlin, C.J. (2017). Budget-Constrained Multi-Armed Bandits with Multiple Plays. arXiv:1711.05928.
- [6] Sakana AI. (2025). AI Scientist-v2: Workshop-Level Automated Scientific Discovery. <https://sakana.ai/ai-scientist>.
- [7] Belz, A., Thomson, R. (2025). The 2025 ReproNLP Shared Task on Reproducibility of Evaluations in NLP. ACL 2025.
- [8] Qian, et al. (2024). Scaling Large Language Model-based Multi-Agent Collaboration. arXiv:2406.07155.
- [9] Kocsis, L., Szepesvari, C. (2006). Bandit Based Monte-Carlo Planning. ECML.
- [10] Jamieson, K., et al. (2014). lil' UCB: An Optimal Exploration Algorithm for Multi-Armed Bandits. COLT.